



ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO THỰC HÀNH 02

Làm trơn ảnh và phát hiện biên cạnh

Nhóm: Myopia
MSHV 1: 25C11067 - Nguyễn Thái Thông
MSHV 2: 25C11035 - Trần Hạ Khánh Duy

TP. Hồ Chí Minh, 2026

1 Mục tiêu

Bài thực hành gồm hai nhóm nội dung chính: làm trơn ảnh và phát hiện biên cạnh. Phần làm trơn ảnh cài đặt và so sánh các bộ lọc mean filter, Gaussian filter và median filter. Phần phát hiện biên cạnh cài đặt và so sánh Sobel, Laplacian và Canny. Mỗi thuật toán được thực hiện bằng hai cách: cài đặt thủ công và dùng hàm OpenCV, sau đó đánh giá bằng hình minh chứng, MSE, thời gian chạy và bộ nhớ sử dụng.

2 Phân công đóng góp

MSHV	Họ tên	Công việc	Tỉ lệ
25C11067	Nguyễn Thái Thông	Cài đặt mean filter, Gaussian filter, Sobel; xây dựng hàm đo thời gian/bộ nhớ, kiểm thử kết quả thủ công so với OpenCV, thu thập hình minh chứng cho phần làm trơn và Sobel.	50%
25C11035	Trần Hạ Khánh Duy	Cài đặt median filter, Laplacian, Canny; tổng hợp số liệu MSE, nhận xét kết quả, lưu output, hoàn thiện notebook và viết báo cáo.	50%

3 Bảng tự đánh giá

Nội dung	Mức độ hoàn thành
Chuẩn bị môi trường, tải ảnh Lenna và hiển thị ảnh	100%
Làm trơn ảnh bằng mean filter	100%
Làm trơn ảnh bằng Gaussian filter	100%
Làm trơn ảnh bằng median filter	100%
Phát hiện biên cạnh bằng Sobel	100%
Phát hiện biên cạnh bằng Laplacian	100%
Phát hiện biên cạnh bằng Canny	100%
So sánh với hàm OpenCV	100%
Thu thập hình minh chứng và lưu kết quả	100%
Báo cáo	100%

4 Cơ sở lý thuyết tóm tắt

4.1 Làm trơn ảnh

Làm trơn ảnh là bước tiền xử lý dùng để giảm nhiễu hoặc giảm các chi tiết nhỏ trong ảnh. Bộ lọc trung bình thay mỗi pixel bằng giá trị trung bình của vùng lân cận nên làm mờ đều. Bộ lọc Gaussian

dùng trọng số lớn hơn cho các điểm gần tâm kernel, vì vậy ảnh sau lọc thường tự nhiên hơn so với mean filter. Bộ lọc median thay pixel bằng trung vị của vùng lân cận, phù hợp khi ảnh có nhiều dạng điểm hoặc nhiễu muối tiêu.

4.2 Phát hiện biên cạnh

Biên cạnh là vùng có thay đổi cường độ sáng mạnh. Sobel dùng đạo hàm bậc một theo hai hướng x và y , sau đó tính độ lớn gradient. Laplacian dùng đạo hàm bậc hai nên nhạy với các vùng biến thiên mạnh nhưng cũng dễ bị ảnh hưởng bởi nhiễu. Canny là phương pháp nhiều bước gồm làm trơn Gaussian, tính gradient, non-maximum suppression, ngưỡng kép và nối biên bằng hysteresis.

5 Đánh giá kết quả

5.1 Ảnh đầu vào

Ảnh Lenna được tải và resize về kích thước nhỏ hơn để các thuật toán thủ công chạy nhanh. Ảnh màu được chuyển sang ảnh xám trước khi áp dụng các thuật toán phát hiện biên.

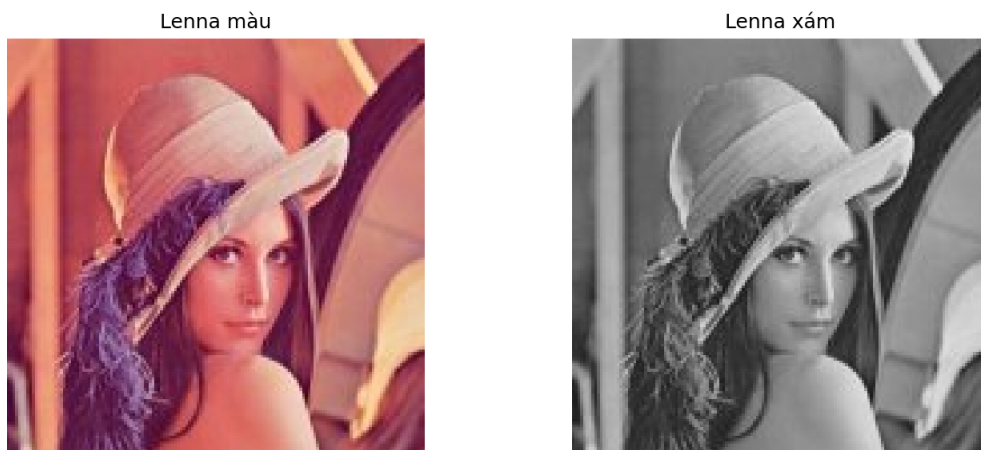


Figure 1: Ảnh Lenna màu và ảnh Lenna xám.

5.2 Làm trơn ảnh

Mean filter làm giảm nhiễu bằng cách lấy trung bình các pixel trong vùng lân cận. Kết quả thủ công và OpenCV có hình dạng trực quan tương tự nhau; sai khác nhỏ có thể đến từ cách xử lý biên và làm tròn giá trị pixel.

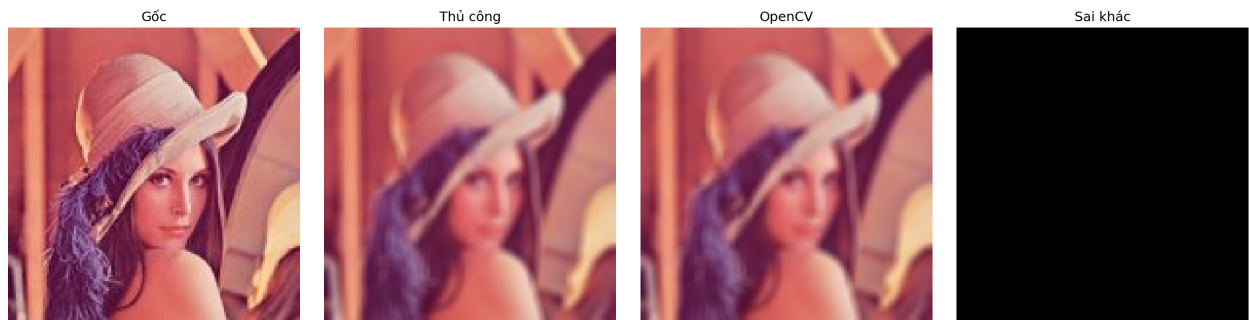


Figure 2: So sánh mean filter thủ công và OpenCV.

Gaussian filter dùng kernel có trọng số theo phân phối Gaussian. So với mean filter, bộ lọc này thường giữ cấu trúc ảnh mềm và tự nhiên hơn vì pixel gần tâm có ảnh hưởng lớn hơn pixel xa tâm.

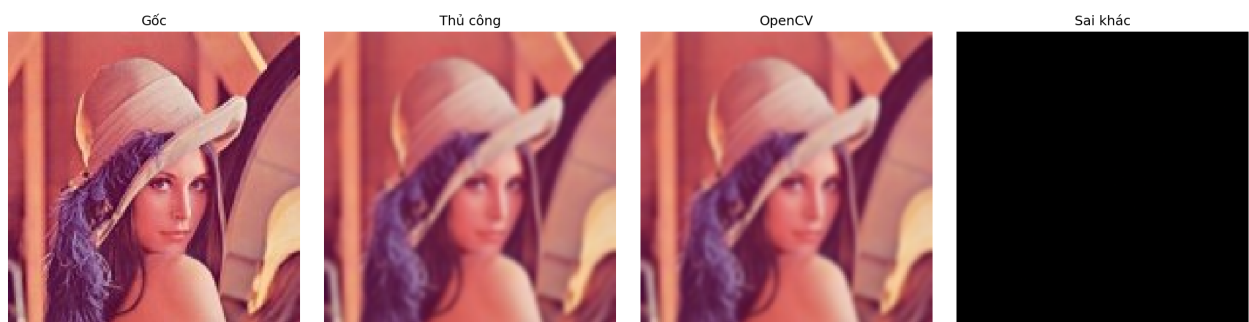


Figure 3: So sánh Gaussian filter thủ công và OpenCV.

Median filter lấy trung vị trong vùng lân cận, do đó có khả năng loại bỏ nhiễu điểm tốt. Kết quả có xu hướng giữ biên tốt hơn mean filter trong một số vùng ảnh.

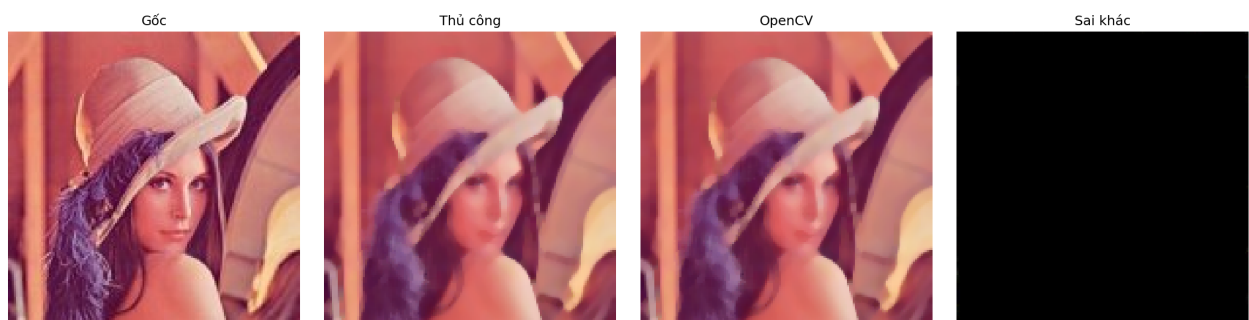


Figure 4: So sánh median filter thủ công và OpenCV.

5.3 Phát hiện biên cạnh

Sobel phát hiện biên bằng cách tính gradient theo hai hướng. Ảnh biên thể hiện rõ các vùng thay đổi cường độ sáng như đường viền khuôn mặt, nón và vai áo trong ảnh Lenna.

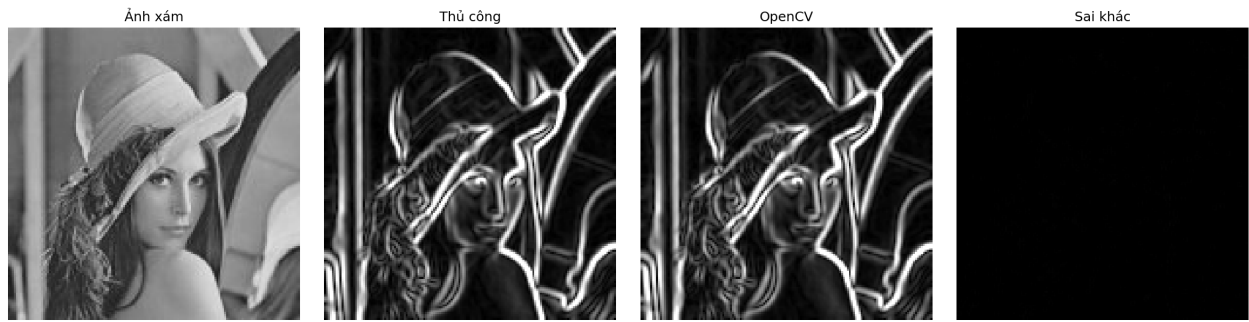


Figure 5: So sánh phát hiện biên Sobel thủ công và OpenCV.

Laplacian dùng đạo hàm bậc hai nên phản ứng mạnh tại vùng có thay đổi cường độ đột ngột. Trước khi áp dụng Laplacian, ảnh được làm trơn Gaussian để giảm bớt nhiễu.

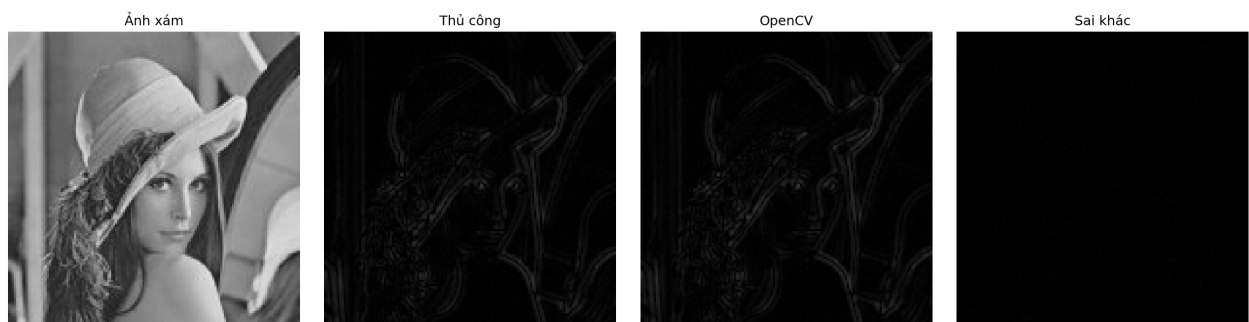


Figure 6: So sánh phát hiện biên Laplacian thủ công và OpenCV.

Canny tạo biên mảnh hơn nhờ bước non-maximum suppression và ngưỡng kép. Kết quả thủ công có thể khác OpenCV nhiều hơn Sobel và Laplacian vì OpenCV tối ưu nhiều chi tiết nội bộ trong quá trình làm mảnh và nối biên.

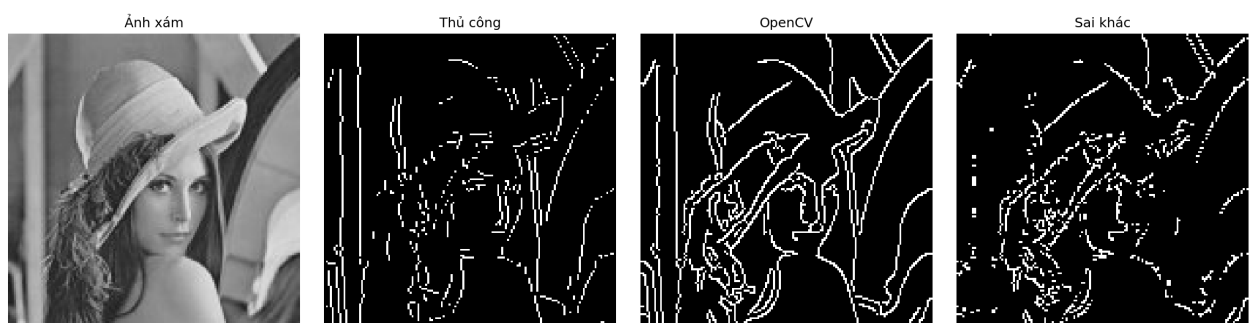


Figure 7: So sánh phát hiện biên Canny thủ công và OpenCV.

5.4 So sánh kết quả

Bảng dưới đây được điền theo kết quả lưu trong file `result.txt` sau khi chạy notebook. Do thời gian và bộ nhớ phụ thuộc vào máy chạy, các giá trị cụ thể nên được lấy từ lần chạy thực nghiệm cuối cùng.

Phép xử lý	MSE	Thủ công	OpenCV	Nhận xét
Mean filter	0.0000	0.359042s 1077.81KB	0.000061s 66.35KB	Hai kết quả gần nhau; sai khác chủ yếu ở xử lý biên và làm tròn.
Gaussian filter	0.0053	0.227804s 801.73KB	0.000117s 66.35KB	Kết quả trực quan tương tự; OpenCV nhanh hơn nhờ tối ưu tích chập.
Median filter	0.4045	0.825825s 217.20KB	0.000165s 66.18KB	Median thủ công chậm hơn do phải tính trung vị cho từng cửa sổ.
Sobel	0.2713	0.498759s 470.88KB	0.000230s 397.21KB	Biên chính được phát hiện rõ; sai khác nhỏ có thể do padding và kiểu dữ liệu.
Laplacian	0.3692	0.335916s 294.42KB	0.000147s 88.54KB	Phản ứng mạnh tại vùng biến thiên cường độ; nhạy với nhiễu nên cần Gaussian trước.
Canny	4629.7798	0.509045s 562.58KB	0.000195s 44.40KB	Sai khác có thể lớn hơn vì Canny gồm nhiều bước và OpenCV có tối ưu nội bộ.

Nhìn chung, cài đặt thủ công giúp hiểu rõ nguyên lý của từng thuật toán, nhưng tốc độ chậm hơn do sử dụng vòng lặp Python. OpenCV phù hợp hơn khi cần xử lý nhanh, ổn định và triển khai thực tế. Trong các kết quả đo được, Mean filter có MSE thấp nhất, còn OpenCV luôn vượt trội về thời gian chạy và bộ nhớ sử dụng.

6 Kết luận

Bài thực hành hoàn thành hai nhóm nội dung: làm trơn ảnh và phát hiện biên cạnh. Các bộ lọc làm trơn giúp giảm nhiễu và chuẩn bị ảnh tốt hơn cho bước phát hiện biên. Các thuật toán phát hiện biên cho thấy nhiều cách tiếp cận khác nhau: Sobel dựa trên gradient bậc một, Laplacian dựa trên đạo hàm bậc hai, còn Canny kết hợp nhiều bước để tạo biên mảnh và rõ hơn.

Kết quả so sánh cho thấy OpenCV nhanh hơn đáng kể so với cài đặt thủ công. Tuy nhiên, phần cài đặt thủ công vẫn cần thiết trong bài thực hành vì giúp kiểm chứng công thức, hiểu rõ xử lý kernel, padding, gradient, ngưỡng và quá trình tạo ảnh biên.